Absolutely. If we're actually going to **execute** this as your project, let's treat it like a real software + ML project rather than just a demo.

■ AI Resume Analyzer — Everything You Need

1. What we're building

Our final system will look roughly like this:

User → Upload Resume + Job Description → AI Analysis → Dashboard

It should produce:

- **ATS Score**
- **Job Match Percentage**
- ■ Matching skills
- ■ Missing skills
- ■ Important keywords
- ■ Skill/category analysis
- ■ Resume improvement suggestions
- ■ Recommended skills to learn
- ■ Downloadable analysis report

---

2. Technologies we'll need

■■ Frontend

I'd recommend:

- **HTML**
- **CSS**
- **JavaScript**
- **React.js**
- **Tailwind CSS**

Since you're already interested in MERN, React will be useful for your portfolio.

■■ Backend

We'll use:

Python + FastAPI

Why FastAPI?

It's lightweight, modern, and works really nicely with Python ML models.

Backend handles:

```
React
  ↓
FastAPI
  ↓
ML Model
  ↓
Result
  ↓
React
```

---

3. Machine Learning / NLP

This is the heart of the project.

We'll need:

Python libraries

```
pandas
numpy
scikit-learn
nltk
spacy
```

For PDF:

```
PyMuPDF
```

Later, optionally:

```
sentence-transformers
```

for semantic similarity.

---

4. What ML concepts you'll need

You don't need to become an NLP expert before starting.

We'll learn/use:

Basic NLP

- Tokenization
- Stopword removal
- Stemming/Lemmatization
- Text cleaning

- Keyword extraction

Machine Learning

- TF-IDF
- Cosine Similarity
- Classification
- Feature extraction
- Model evaluation

Advanced version

- Word embeddings
- Sentence embeddings
- BERT / Sentence-BERT
- Semantic similarity

---

5. Dataset

This is something we need to plan carefully.

We'll potentially need datasets containing:

Resume data

Examples:

```
Resume → Skills → Job Category
```

and/or

Job descriptions

```
Job Description → Required Skills → Job Category
```

We can use public datasets from sources such as Kaggle.

But there's an important point:

We don't necessarily need a giant dataset for the first version.

Our first matching system can use:

> Resume text + Job Description → TF-IDF → Cosine Similarity

Then we can improve it.

---

6. Skill database

This will make the project MUCH better.

We'll create something like:

```
skills.json
```

containing categories such as:

```
Programming:
Python
Java
C++
JavaScript

Web:
HTML
CSS
React
Node.js
Express

Database:
MongoDB
MySQL
PostgreSQL

ML:
Machine Learning
Deep Learning
TensorFlow
PyTorch
Scikit-learn

Cloud:
AWS
Azure
Docker
Kubernetes
```

Then our program can identify:

Resume:

> Python, React, MongoDB, TensorFlow

Job requires:

> Python, React, AWS, Docker, MongoDB

Result:

```
Matched:
Python
React
MongoDB

Missing:
AWS
Docker
```

---

7. Resume processing

The user uploads:

```
resume.pdf
```

We'll extract the text.

Example:

```
Samiha Tabassum
Electronics Engineer

Skills:
Python
Machine Learning
HTML
CSS
React
```

The PDF parser converts that into plain text.

Then:

```
PDF
 ↓
Text
 ↓
Cleaning
 ↓
Tokenization
 ↓
Skill extraction
 ↓
TF-IDF
 ↓
Similarity
```

---

8. ATS scoring

We'll create our own scoring system.

For example:

ComponentWeight	
Job similarity	40%
Required skills	30%
Keywords	15%
Resume sections	10%
Formatting/basic checks	5%

Then:

```
ATS Score = weighted combination
```

This doesn't mean we're reproducing a proprietary ATS. We're creating **our own ATS-style scoring system**.

That's actually better for an academic project because we can explain exactly how our score is calculated.

---

9. Job matching

This is one of the most important parts.

Suppose the job description says:

> Looking for a Python developer experienced in Django, REST API, PostgreSQL and Docker.

Resume says:

> Python developer with experience in Flask, REST API and MySQL.

Our system should understand:

Match

- Python
- REST API

Missing

- Django
- PostgreSQL
- Docker

And calculate a match score.

---

10. Dashboard

This is where your project starts looking professional.

The result page could have:

```
RESUME ANALYSIS

■■■■■■■■■■■■■■■■■
■ ATS SCORE    ■
■     78%      ■
■■■■■■■■■■■■■■■■■

JOB MATCH: 84%

Matched Skills
✓ Python
✓ React
✓ SQL

Missing Skills
✗ Docker
✗ AWS

Resume Strength
■■■■■■■■■■ 78%

Recommendations
```

```
→ Add measurable achievements
→ Add Docker experience
→ Improve skills section
```

---

11. Pages we'll need

1■■ Landing Page

```
AI Resume Analyzer

Analyze your resume.
Match it with jobs.
Improve your chances.

[Analyze Resume]
```

2■■ Login/Register

Optional initially.

3■■ Upload Page

```
Upload Resume
[ Choose PDF ]

Paste Job Description
[                    ]

[ Analyze ]
```

4■■ Analysis Page

Shows:

- ATS score
- Match score
- Skills
- Missing skills
- Keywords
- Suggestions

5■■ History

Shows previous analyses.

6■■ Profile

Optional.

---

12. Database

If we want accounts/history:

MongoDB

We'll store:

```
User
 ■■■ name
 ■■■ email
 ■■■ password

Analysis
 ■■■ user_id
 ■■■ resume
 ■■■ job_description
 ■■■ ats_score
 ■■■ match_score
 ■■■ matched_skills
 ■■■ missing_skills
 ■■■ date
```

We don't need MongoDB for the first prototype.

---

13. API endpoints

Our backend might eventually have:

```
POST /upload-resume
POST /analyze
POST /extract-skills
POST /calculate-score
GET  /analysis-history
GET  /analysis/{id}
```

React communicates with FastAPI through these APIs.

---

14. Project folder structure

Eventually something like:

```
AI-Resume-Analyzer/
■
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ services/
■   ■   ■■■ App.jsx
■   ■■■ package.json
■
■■■ backend/
■   ■■■ main.py
■   ■■■ routes/
■   ■■■ models/
■   ■■■ services/
■   ■■■ requirements.txt
■
■■■ ml/
■   ■■■ preprocessing.py
■   ■■■ skill_extraction.py
■   ■■■ similarity.py
```

```
■   ■■■ scoring.py
■   ■■■ model.py
■
■■■ data/
■   ■■■ resumes/
■   ■■■ job_descriptions/
■   ■■■ skills.json
■
■■■ notebooks/
■   ■■■ experiments.ipynb
■
■■■ README.md
■■■ .gitignore
```

Don't worry if this looks huge ■. **We won't create all of this on day one.**

---

15. Software you'll need

You can do almost everything on your laptop.

Install:

Essential

- **VS Code**
- **Python**
- **Node.js**
- **Git**
- **GitHub**
- **MongoDB** (later)

You already use VS Code, Colab and GitHub, so you're not starting from zero.

---

16. Where we'll code

ML experimentation

Google Colab

Good for:

- Dataset exploration
- NLP experiments
- Model testing
- Visualization

Then move the finished Python code into VS Code.

Website/backend

VS Code

Version control

GitHub

---

17. APIs

We don't necessarily need external APIs initially.

But later we could optionally use an LLM API for things like:

> "Give personalized suggestions to improve this resume."

However, **I would NOT make an external AI API the core of your project.**

Your actual ML/NLP system should work independently.

That makes the project academically stronger.

---

18. Advanced features

If the basic version works, we can add:

■ Semantic matching

Instead of just matching exact words:

> "Machine learning engineer"

and

> "AI/ML developer"

can be recognized as semantically related.

Using:

Sentence-BERT

■ Resume section detection

Identify:

```
Education
Experience
Skills
Projects
Certifications
Achievements
```

Then tell the user what's missing.

■ Job recommendations

If the resume has:

```
Python
Pandas
Scikit-learn
SQL
```

the system could recommend:

```
Data Analyst
ML Intern
Junior Data Scientist
```

■ Resume improvement

For example:

Bad:

> Worked on machine learning project.

Better suggestion:

> Developed a machine learning model using Python and Scikit-learn to classify...

---

19. Evaluation — VERY important for your course

Don't just say:

> "Our system works."

We need measurable results.

For the ML component we can evaluate:

- Accuracy
- Precision

- Recall
- F1-score

For similarity/matching:

- Similarity score
- Precision of skill extraction
- Correct vs incorrect matches

And compare models.

For example:

```
                    F1 Score

Naive Bayes          82%
Logistic Reg.        87%
SVM                  89%
```

If we use semantic embeddings later:

```
TF-IDF               78%
Sentence-BERT        91%
```

Only report numbers we actually obtain from experiments.

---

20. What you'll need to learn

You don't need to learn everything beforehand.

We'll learn it as we build.

Priority 1

You already have some of this:

- Python
- Pandas
- Scikit-learn
- ML basics
- Git/GitHub
- HTML/CSS

Priority 2

Learn:

- NLP basics
- TF-IDF
- Cosine similarity

- PDF text extraction
- FastAPI
- REST APIs
- React basics

Priority 3

Later:

- MongoDB
- Sentence-BERT
- Authentication
- Deployment

---

21. Project documentation

For your university submission, we'll eventually need:

■ Proposal

- Introduction
- Problem statement
- Objectives
- Methodology
- Expected outcome

■ SRS

Software Requirements Specification.

■ Diagrams

- System architecture
- Use-case diagram
- Flowchart
- ER diagram
- Data-flow diagram

■ ML documentation

- Dataset
- Preprocessing
- Feature extraction
- Algorithms

- Training/testing
- Evaluation

■ Final report

- Introduction
- Literature review
- Methodology
- Implementation
- Results
- Discussion
- Conclusion
- Future work

■ Presentation

We'll make the demo/presentation around the actual system.

---

22. The actual development roadmap

This is the important part.

Don't try to build the entire thing immediately.

Phase 0 — Planning

Day 1

- Finalize features
- Finalize tech stack
- Find dataset
- Design architecture

↓

Phase 1 — ML Prototype

Days 2–5

Build:

```
Resume
 ↓
Text extraction
 ↓
Preprocessing
 ↓
TF-IDF
 ↓
Cosine similarity
```

```
        ↓
Match score
```

↓

Phase 2 — Skill Extraction

Days 6–8

Build:

```
Resume → Skills
Job → Required Skills
            ↓
      Compare them
```

↓

Phase 3 — FastAPI

Days 9–11

Connect ML → backend.

↓

Phase 4 — React

Days 12–16

Build the actual website.

↓

Phase 5 — Database

Days 17–18

Add users + history.

↓

Phase 6 — Improvements

Days 19–23

Add:

- Better UI
- Better scoring
- Semantic similarity
- Suggestions
- Charts

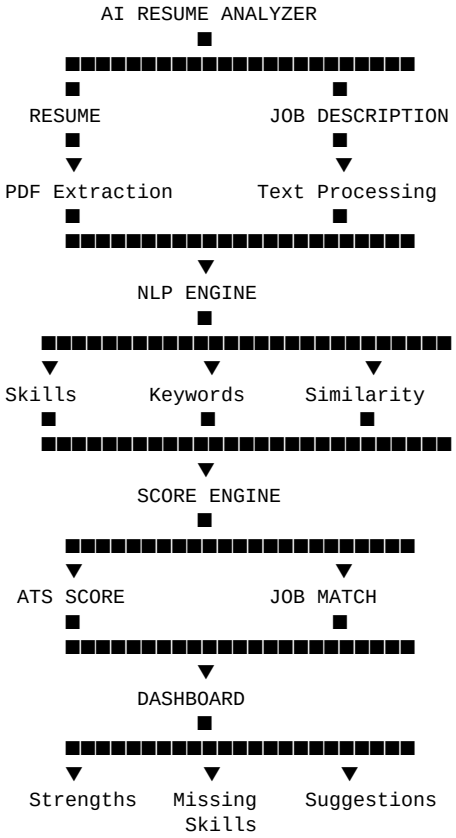
↓

Phase 7 — Testing & Documentation

Days 24–30

- Test
- Evaluate
- Screenshots
- Report
- Presentation
- GitHub README

■ What I'd consider the "final version"



One important recommendation

Don't start with React.

I know the website is probably the exciting part ■, but if we start there, you'll end up with a beautiful page that doesn't actually analyze anything.

We'll start by making the **ML engine work in Google Colab with one resume and one job description**.

Once we can genuinely say:

> "Give me these two texts → my system calculates a meaningful match score and identifies skills."

then we'll turn it into the website.

That gives us a solid project underneath the pretty UI—and makes your viva much easier because you'll actually understand every part.